



Politecnico di Torino

Microelectronic Systems

DLX Microprocessor: Design & Development

Final Project Report

Master degree in Electronics Engineering
Master degree in Computer Engineering

Referents: Prof. Mariagrazia Graziano, Giovanna Turvani

Authors: Group ms25.37

Silvia Bonenti - s339232

July 25, 2025

Contents

1	Introduction	1
1.1	Pipeline Architecture	1
1.2	Instruction Set	1
1.2.1	Architecture	1
1.2.2	Enumeration	3
1.2.3	Coding	4
1.3	Structure	5
1.3.1	Block Diagram	5
2	Datapath	7
2.1	Overview	7
2.2	Block Diagram	7
2.3	ALU	8
2.3.1	Block Diagram	8
2.3.2	Arithmetic Unit	8
2.3.3	Logical Unit	9
2.3.4	Shift Unit	10
2.3.5	Comparison Unit	11
2.4	Register File	11
2.5	Pipeline Stages	11
2.5.1	Instruction Fetch (IF)	12
2.5.2	Instruction Decode (ID)	12
2.5.3	Execute (EX)	12
2.5.4	Memory Access (MEM)	12
2.5.5	Write Back (WB)	13
2.6	Control Signals	13
2.7	Forwarding	13
2.8	Stalls	13
2.8.1	Example	13
2.9	Control Hazards	14
3	Control Unit	15
3.1	Overview	15
3.2	Control Unit Design	15
3.3	Block Diagram	16
3.4	Control Signals	16
3.5	Lookup Table (LUT)	17
3.5.1	LUT Encoding	17
3.6	ALU Operation Code Generation	17
3.7	Flush and Stall Operations	17
3.7.1	Flush Operation	18
3.7.2	Stall Operation	18

4	Memories	19
4.1	Overview	19
4.2	Instruction Memory	19
4.2.1	Block Diagram	19
4.3	Data Memory	20
4.3.1	Block Diagram	20
4.4	Memory Initialization	21
5	Simulation	22
5.1	Overview	22
5.2	Testbench Design	22
5.2.1	Block Diagram	22
5.2.2	ALU's Testbench	23
5.3	Simulation Tool	23
5.3.1	Compilation and Simulation Scripts	23
5.4	Assembler	24
5.4.1	Assembler Script	24
5.5	Test Programs	24
5.5.1	Test Program 1: Loop Handling	24
5.5.2	Test Program 2: Complex Scenario	24
5.5.3	Test Program 3: Stalls and Forwarding	25
6	Synthesis	26
6.1	Overview	26
6.2	Synthesis Tool	26
6.3	Scripts	26
6.3.1	Synthesis Bash Script	26
6.3.2	Synthesis TCL Scripts	26
6.3.3	Synthesis SDC File	27
6.4	Synthesis Results	27
6.5	Analysis of Results	28
6.5.1	Timing Analysis	28
6.5.2	Area Analysis	28
6.5.3	Power Analysis	28
7	Physical Design	29
7.1	Overview	29
7.2	Physical Design Tool	29
7.3	Physical Design Flow	29
7.4	Schematic	30
7.5	Control Unit Results	31
7.5.1	Area	31
7.5.2	Power	31
7.5.3	Timing Analysis	31
7.6	Datapath Results	32
7.6.1	Area	32
7.6.2	Power	32
7.6.3	Timing Analysis	33
7.7	Conclusion	33
7.8	Comparison of Synthesis vs Physical Design Results	33
7.8.1	Control Unit Comparison	33
7.8.2	Datapath Comparison	34
7.8.3	Summary	34

8	Conclusions and Further Implementations	35
8.1	Overview	35
8.2	Key Findings	35
8.3	Further Implementations	35
A	Simulation	37
A.1	Subappendix 1: Compilation Script	37
A.2	Subappendix 2: Simulation Script	38
A.3	Subappendix 3: Assembler Script	38
A.4	Subappendix 4: Test Programs	39
A.4.1	Test Program 1	39
A.4.2	Test Program 2	39
A.4.3	Test Program 3	39
B	Synthesis Scripts	41
B.1	Subappendix 1: Synthesis Script	41
B.2	Subappendix 2: Synthesis TCL Script	41
B.3	Subappendix 3: Synthesis SDC Script	42

CHAPTER 1

Introduction

The DLX (or DELUXE) architecture is an example of a RISC (Reduced Instruction Set Computer) architecture, developed by John L. Hennessy and David A. Patterson primarily for educational purposes.

It is designed to be simple and easy to understand, making it a very good option for teaching computer architecture concepts.

DLX features a byte-addressable memory organized in big-endian format, with a 32-bit address space that supports up to 4 GB of memory. Its data path and instructions are both 32 bits wide. It includes:

- 32 general-purpose registers, each 32 bits wide,
- A 32-bit program counter (PC),
- A set of 32 floating-point registers, also 32 bits wide,
- A set of special-purpose registers, such as the status register and the exception register.

1.1 Pipeline Architecture

The DLX operates on a 5-stage pipeline architecture, which allows to execute multiple instructions simultaneously, improving performance. The stages of the pipeline are:

- Instruction Fetch (IF): The instruction is fetched from memory.
- Instruction Decode (ID): The instruction is decoded and the operands are fetched from the registers.
- Execute (EX): The operation specified by the instruction is performed and the condition flags are updated.
- Memory Access (MEM): If the instruction involves memory access, the data is read from or written to memory.
- Write Back (WB): The result of the operation is written back to the destination register.

1.2 Instruction Set

1.2.1 Architecture

The architecture supports a variety of instruction types, in particular R-type (register), I-type (immediate), and J-type (jump) instructions. In particular:

- R-type instructions are used for arithmetic and logical operations between registers.

- I-type instructions are used for operations that involve an immediate value, such as loading/storing a value from/to memory, performing an arithmetic operation with an immediate value, and conditional branch instructions.
- J-type instructions are used for jump and jump and link operations, allowing the program to change its execution flow.

The encoding of the DLX instruction set is designed to be simple and efficient, with a fixed length of 32 bits for each instruction. The opcode field is used to identify the type of instruction and the operation to be performed, while the other fields are used to specify the operands and immediate values. The encoding is designed to minimize the number of bits required for each instruction, while still allowing for a wide range of operations and immediate values. The DLX instruction set encoding is shown in Figure 1.1.

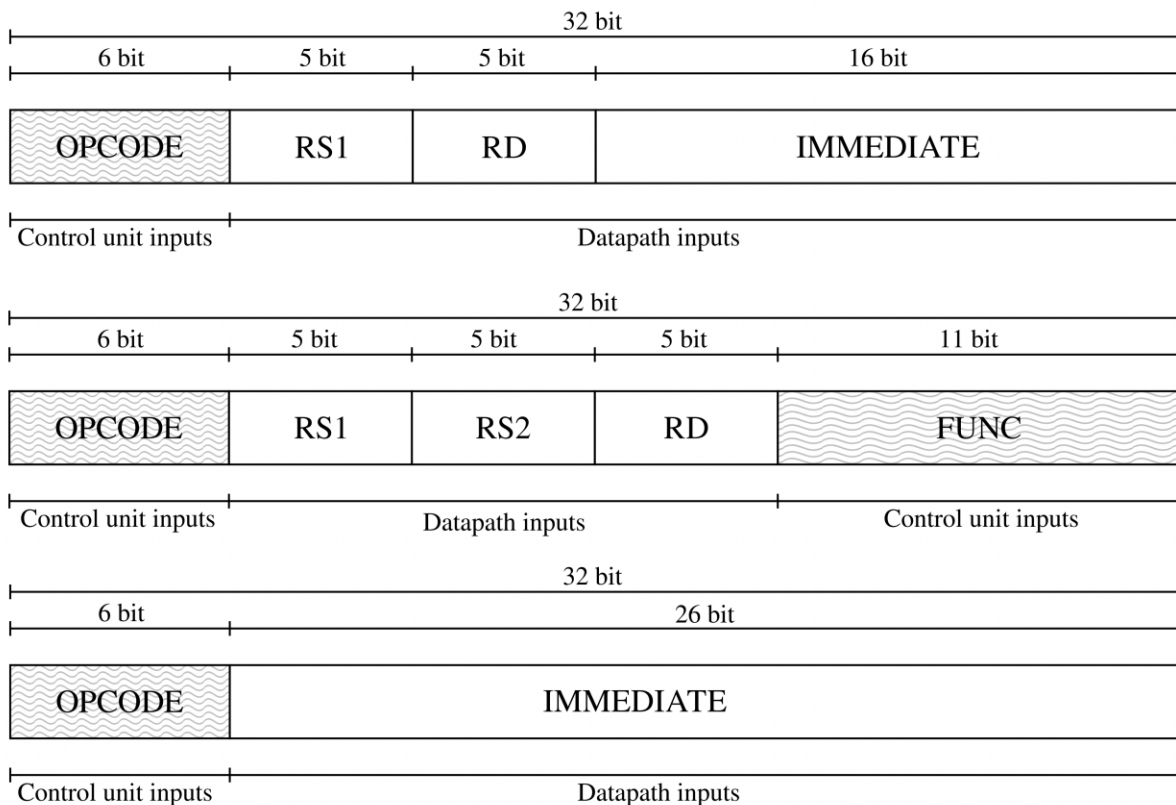


Figure 1.1: DLX Instruction Set Encoding. The figure shows the different types of instructions and their fields.

R type Instructions

R-type instructions are characterized by the fact that all operands are registers, and they do not involve immediate values or memory access. They are composed of the following fields:

- **opcode:** 6 bits that specify the operation to be performed
- **rs1:** 5 bits that specify the first source register
- **rs2:** 5 bits that specify the second source register
- **rd:** 5 bits that specify the destination register
- **func:** 6 bits that specify the function

I type Instructions

I-type instructions are characterized by the fact that they involve an immediate value or memory access. They are composed of the following fields:

- **opcode:** 6 bits that specify the operation to be performed
- **rs1:** 5 bits that specify the source register
- **rd:** 5 bits that specify the destination register
- **imm:** 16 bits that specify the immediate value or the offset for memory access

J type Instructions

J-type instructions are used for jump operations and are composed of the following fields:

- **opcode:** 6 bits that specify the operation to be performed
- **imm:** 26 bits that specify the target address for the jump operation

1.2.2 Enumeration

The DLX instruction set in its basic form includes the following instructions:

- **add, sub, and, or, xor, sll, srl, sne, sle, sge:** R-type arithmetic and logical operations.
- **addi, andi, ori, xori, slli, srli, snei, slei, sgei:** I-type arithmetic and logical operations with immediate values.
- **lw, sw:** I-type load and store instructions for memory access.
- **beqz, bnez:** I-type conditional branch instructions.
- **j, jal:** J-type jump instructions.
- **nop:** No operation instruction.

R-type Instructions

- **add RA, RB, RC** (meaning $R[RA] = R[RB] + R[RC]$)
- **sub RA, RB, RC** (meaning $R[RA] = R[RB] - R[RC]$)
- **and RA, RB, RC** (meaning $R[RA] = R[RB] \& R[RC]$)
- **or RA, RB, RC** (meaning $R[RA] = R[RB] | R[RC]$)
- **xor RA, RB, RC** (meaning $R[RA] = R[RB] \oplus R[RC]$)

- `sll RA, RB, RC` (meaning $R[RA] = R[RB] \ll R[RC]$)
- `srl RA, RB, RC` (meaning $R[RA] = R[RB] \gg R[RC]$)
- `sne RA, RB, RC` (meaning $R[RA] = (R[RB] \neq R[RC]) ? 1 : 0$)
- `sle RA, RB, RC` (meaning $R[RA] = (R[RB] \leq R[RC]) ? 1 : 0$)
- `sge RA, RB, RC` (meaning $R[RA] = (R[RB] \geq R[RC]) ? 1 : 0$)

I-type Instructions

- `addi RA, RB, #imm` (meaning $R[RA] = R[RB] + \text{imm}$)
- `subi RA, RB, #imm` (meaning $R[RA] = R[RB] - \text{imm}$)
- `andi RA, RB, #imm` (meaning $R[RA] = R[RB] \& \text{imm}$)
- `ori RA, RB, #imm` (meaning $R[RA] = R[RB] | \text{imm}$)
- `xori RA, RB, #imm` (meaning $R[RA] = R[RB] \oplus \text{imm}$)
- `slli RA, RB, #imm` (meaning $R[RA] = R[RB] \ll \text{imm}$)
- `srli RA, RB, #imm` (meaning $R[RA] = R[RB] \gg \text{imm}$)
- `snei RA, RB, #imm` (meaning $R[RA] = (R[RB] \neq \text{imm}) ? 1 : 0$)
- `slei RA, RB, #imm` (meaning $R[RA] = (R[RB] \leq \text{imm}) ? 1 : 0$)
- `sgei RA, RB, #imm` (meaning $R[RA] = (R[RB] \geq \text{imm}) ? 1 : 0$)
- `lw RA, #imm(RB)` (meaning $R[RA] = \text{Mem}[RB + \text{imm}]$)
- `sw #imm(RB), RA` (meaning $\text{Mem}[RB + \text{imm}] = R[RA]$)
- `beqz RA, #imm` (meaning if $R[RA] == 0$ then $PC = PC + \text{imm}$)
- `bnez RA, #imm` (meaning if $R[RA] \neq 0$ then $PC = PC + \text{imm}$)

J-type Instructions

- `j #imm` (meaning $PC = \text{imm}$)
- `jal #imm` (meaning $R[31] = PC + 4; PC = PC + \text{imm}$)

1.2.3 Coding

The compiler translates the high-level instructions into machine code by encoding the instructions according to the DLX instruction set architecture. In the following table (1.1), we summarize the `opcode` and `func` encoding for the selected DLX instructions.

Mnemonic	Opcode	Func
sll	0x00	0x04
srl	0x00	0x06
add	0x00	0x20
sub	0x00	0x22
and	0x00	0x24
or	0x00	0x25
xor	0x00	0x26
sne	0x00	0x29
sle	0x00	0x2C
sge	0x00	0x2D
j	0x02	/
jal	0x03	/
beqz	0x04	/
bnez	0x05	/
addi	0x08	/
subi	0x0A	/
andi	0x0C	/
ori	0x0D	/
xori	0x0E	/
slli	0x14	/
nop	0x15	/
srli	0x16	/
snei	0x19	/
slei	0x1C	/
sgei	0x1D	/
lw	0x23	/
sw	0x2B	/

Table 1.1: Opcode encoding for selected DLX instructions.

1.3 Structure

The DLX is composed of several components that work together to execute instructions. The main components of the DLX architecture include:

- Datapath: This is the core of the DLX architecture, responsible for executing instructions and performing arithmetic and logical operations.
- Control Unit: This component generates control signals that coordinate the operation of the datapath and other components.
- IRAM: The Instruction Memory, which stores the instructions to be executed.
- DRAM: The Data Memory, which stores the data used by the instructions.

1.3.1 Block Diagram

The block diagram of the DLX architecture is shown in Figure 1.2. This diagram illustrates the connections between the various components, including the datapath, control unit, IRAM, and DRAM. The arrows indicate the flow of data between the components.

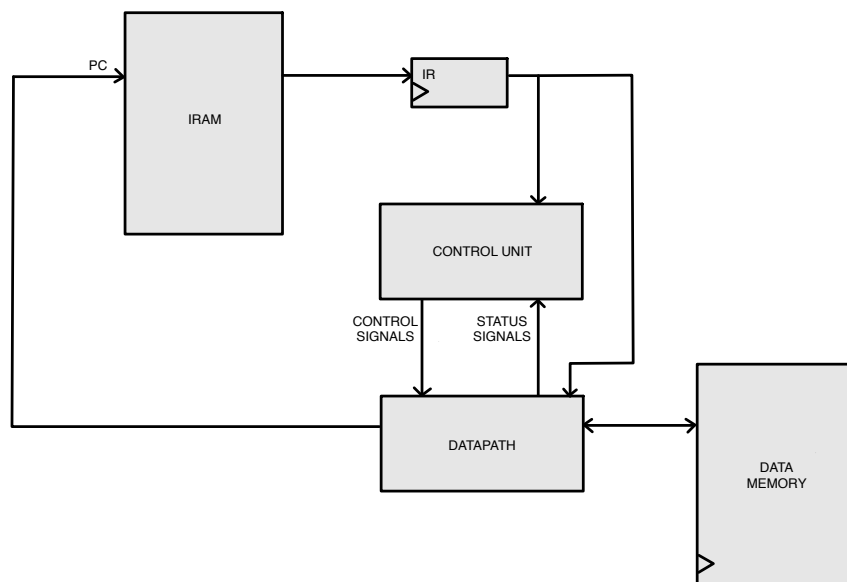


Figure 1.2: Block diagram of the DLX architecture.

CHAPTER 2

Datapath

2.1 Overview

The DLX datapath is designed to support the execution of the DLX instruction set architecture. It consists of several components that work together to fetch, decode, and execute instructions. The main components of the DLX datapath include:

- ALU: The Arithmetic Logic Unit, which performs arithmetic and logical operations.
- Register File: A set of registers used to store operands and results.

2.2 Block Diagram

The block diagram of the DLX datapath is shown in Figure 2.1. This diagram illustrates the connections between the various components, including the ALU and the register file.

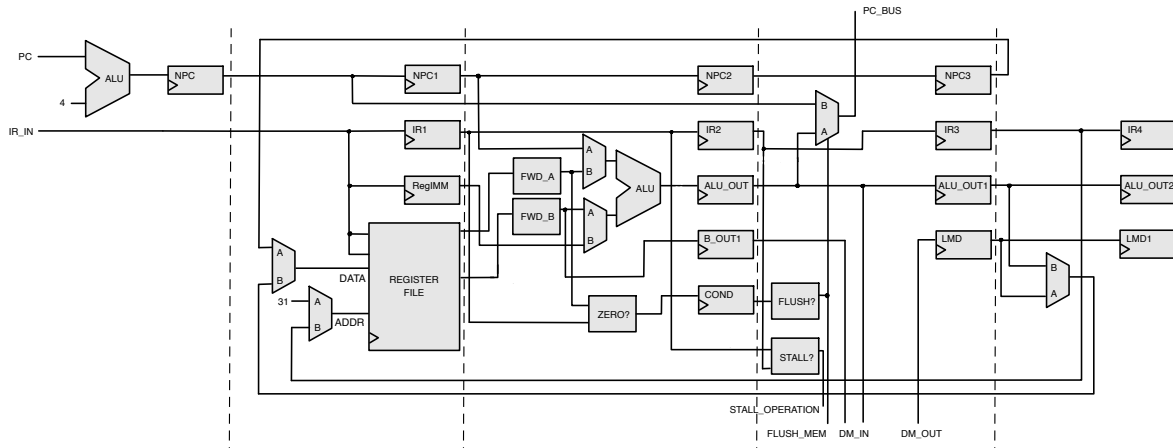


Figure 2.1: Block diagram of the DLX datapath.

2.3 ALU

The ALU is the core component of the DLX datapath, responsible for performing arithmetic and logical operations. It takes two operands from the register file and performs the specified operation, such as addition, subtraction, comparison, logical AND/OR/XOR or shifting. The result is then written back to the register file or sent to memory, depending on the instruction.

The ALU is controlled by a signal called **ALU_OPCODE**, which determines the operation to be performed. The ALU can also handle immediate values, which are constants embedded in the instruction.

It is composed of several functional units, including:

- **Arithmetic Unit:** Performs addition (and subtraction) operations.
- **Logical Unit:** Performs logical operations such as AND, OR, and XOR.
- **Shift Unit:** Performs left and right shifts on the operands.
- **Comparison Unit:** Compares two operands and sets condition flags.

2.3.1 Block Diagram

The block diagram of the ALU is shown in Figure 2.2. This diagram illustrates the internal structure of the ALU, including the functional units and the connections between them.

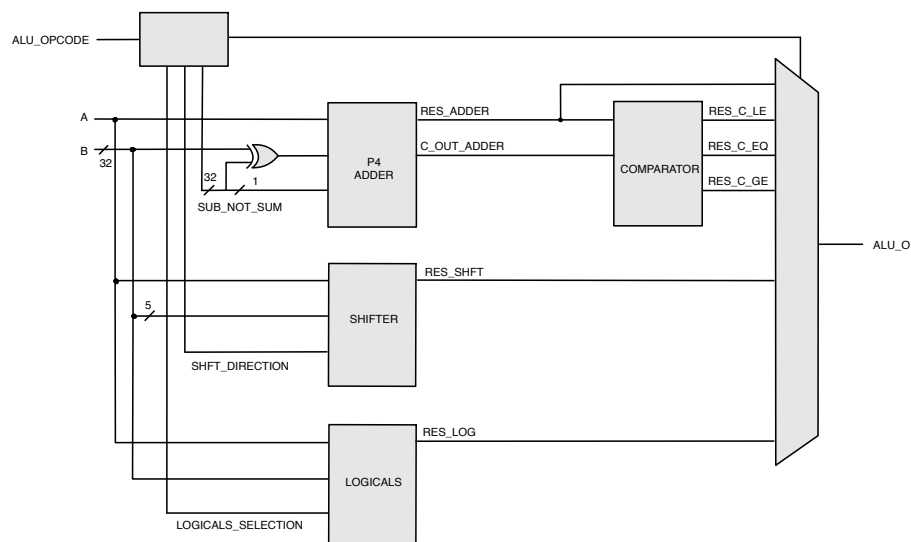


Figure 2.2: Block diagram of the ALU.

2.3.2 Arithmetic Unit

The Arithmetic Unit is obtained simply by the P4 Adder implementation. Specifically, it is composed of two main units:

- **CLA - Sparse Tree Carry Generator:** in charge of computing one carry every four, to speed up computation and reduce the critical path.
- **Carry Select - Like Sum Generator:** Composed of $N_{BIT}/N_{BIT_PER_BLOCK}$ blocks (8 in the DLX), each of which is composed of two Ripple Carry Adders (RCAs), in charge of computing the sum in parallel, one with carry in equal to 0 and the other having it equal to 1. The correct result between the two is selected using the carry in generated by the Tree Carry Generator.

The subtraction is performed by changing the external connections to the P4 Adder component; this is done by transforming the B input in its two's complement equivalent: using the `SUB_NOT_SUM` signal, which is equal to 1 in case of a subtraction and 0 in case of an addition, we perform an XOR between the signal and the input B and we use it as the carry in. As a consequence:

- `SUB_NOT_SUM = 0`: $ALU_O = A + (B \oplus SUB_NOT_SUM) + SUB_NOT_SUM = A + B + 0$
- `SUB_NOT_SUM = 1`: $ALU_O = A + (B \oplus SUB_NOT_SUM) + SUB_NOT_SUM = A + NOT(B) + 1$

Block Diagram

The block diagram of the Arithmetic Unit is shown in Figure 2.3. This diagram illustrates the internal structure of the Arithmetic Unit, including the functional units and the connections between them.

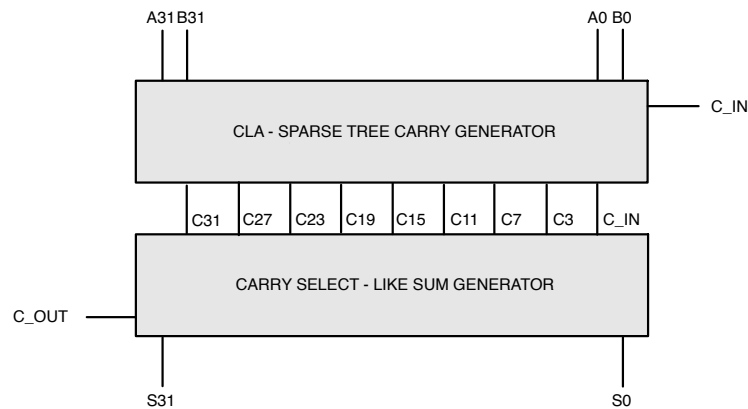


Figure 2.3: Block diagram of the Arithmetic Unit.

2.3.3 Logical Unit

The logical unit is necessary to perform logical operations like AND, OR, XOR. The main goal of this design is to have an implementation in which the delay for computing each of the logical operations is the same.

It is realized as an interconnection of NAND gates and the signals `S0` to `S3` are used to select the needed logical function:

- `A AND B`: `S0=S1=S2 = 0`, `S3 = 1`
- `A OR B`: `S0 = 0`, `S1=S2=S3 = 1`
- `A ⊕ B`: `S0=S3 = 0`, `S1=S2 = 1`

Schematic Diagram

The schematic diagram of the Logical Unit is shown in Figure 2.4. This diagram illustrates the internal structure of the Logical Unit, in particular the NAND gates and the input signals.

Beware: this schematic represents the logical unit only for 1 bit. The structure must be replicated to get the logical operations on more bits.

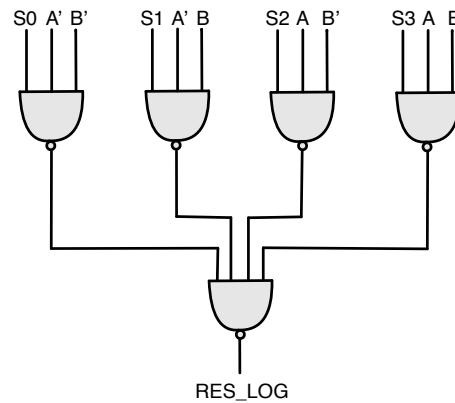


Figure 2.4: Schematic diagram of the Logical Unit.

2.3.4 Shift Unit

The Shift Unit is used to perform a left or right shift of the operand A by an amount specified by the operand B. It is described following the T2 implementation, in which the shifting operation is performed going through 3 steps:

- Masks Generation: 4 masks are generated, for 4 fixed shifting values (0, 8, 16, 24), depending on the direction (left or right) and type (arithmetical or logical) of the shift.
- Coarse Grain Shift: choose the correct mask between the 4 depending on the two MSBs of operand B, which are used as selection signal for the MUX.
- Fine Grain Shift: shift bit by bit the choosen mask by an amount specified by the three LSBs of operand B.

Block Diagram

The block diagram of the Shift Unit is shown in Figure 2.5. This diagram illustrates the internal structure of the Shift Unit, in particular the components and the connections between them.

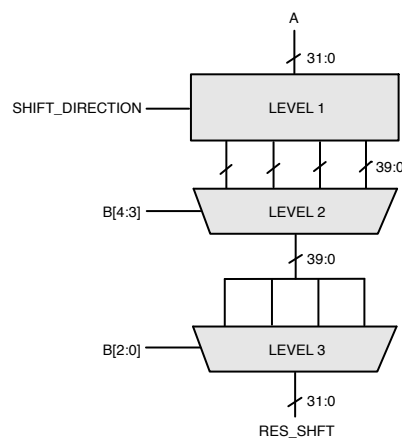


Figure 2.5: Block diagram of the Shift Unit.

2.3.5 Comparison Unit

The Comparison Unit is used to perform comparison operations; in particular, starting from the subtraction between operand A and operand B, which gives as a result SUB and C_OUT (borrow) signals, it performs the following comparisons:

- Less or equal ($\leq$): NOT(C_OUT) OR Z
- Less ($<$): NOT(C_OUT)
- Greater ($>$): C_OUT AND NOT(Z)
- Greater or equal ($\geq$): C_OUT
- Equal ($=$): Z

Schematic Diagram

The schematic diagram of the Comparison Unit is shown in Figure 2.6. This diagram illustrates the internal structure of the Comparison Unit, in particular the logic gates and their interconnections.

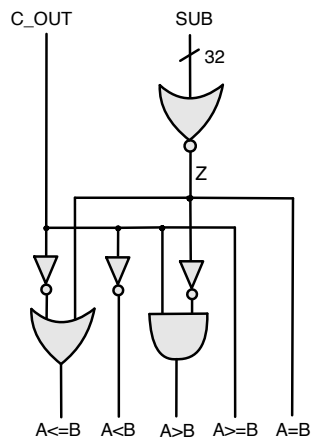


Figure 2.6: Schematic diagram of the Comparison Unit.

2.4 Register File

The register file is a set of registers used to store operands and results. It consists of 32 registers, each capable of holding a 32-bit value. The register file is accessed by the ALU and other components of the datapath to read and write data. The register file also includes control signals to select which registers to read from or write to during the execution of an instruction. It has two read ports and one write port, allowing simultaneous reading of two registers and writing to one register.

2.5 Pipeline Stages

The DLX datapath is designed to support pipelining, which allows multiple instructions to be executed simultaneously by breaking down the execution process into several stages. The main stages of the DLX pipeline are:

- Instruction Fetch (IF): The instruction is fetched from memory and loaded into the instruction register (IR).
- Instruction Decode (ID): The instruction is decoded, and the operands are read from the register file. The control signals for the instruction are also generated during this stage.

- **Execute (EX):** The ALU performs the operation specified by the instruction using the operands read from the register file. The result is computed and sent to the next stage.
- **Memory Access (MEM):** If the instruction involves memory access, the data is read from or written to memory. This stage is used for load and store instructions.
- **Write Back (WB):** The result of the instruction is written back to the register file. This stage is used for R-type and I-type instructions.

The pipeline allows for overlapping execution of instructions, where each stage can be processing a different instruction simultaneously. This improves the overall throughput of the processor and allows for more efficient use of resources.

2.5.1 Instruction Fetch (IF)

In the Instruction Fetch stage, the next instruction is fetched from memory (IRAM) and loaded into the instruction register (IR). The Program Counter (PC) is incremented by 4 and stored in NPC to point to the next instruction. The instruction is then passed to the Instruction Decode stage for further processing.

2.5.2 Instruction Decode (ID)

In the Instruction Decode stage, the instruction in the IR register is decoded to determine the operation to be performed. The operands required for the instruction are read from the register file using the source register addresses specified in the instruction (RS1, RS2 fields). The immediate value is also extracted from the instruction if it is of I-type or J-type and stored in the RegIMM register.

2.5.3 Execute (EX)

In the Execute stage, the ALU performs the operation specified by the instruction, specifically by the ALU_OPCODE control signal. The ALU takes the operands read from the register file and performs the operation, such as addition, subtraction, logical AND/OR/XOR, or comparison. The result of the operation is stored in the ALU_OUT register. If the instruction is a branch or jump instruction, the target address is calculated based on the immediate value and the NPC value.

There is also a forwarding logic that allows the ALU to use the result of a previous instruction directly, without waiting for it to be written back to the register file.

In this stage, also the zero condition is checked, which is used for branch instructions (BNEZ, BEQZ). If the value at the output of the forwarding logic of operand A is either zero or different from zero, depending on the specific branch instruction, the Cond signal is asserted to indicate whether the branch should be taken or not.

The value at the output of the forwarding logic of operand B is then stored in a register called B_OUT1, which is used in the Memory Access stage as the input data.

2.5.4 Memory Access (MEM)

In the Memory Access stage, if the instruction is a load or store instruction, the data is read from or written to memory. The address for the memory operation is calculated using the output of the forwarding logic of operand A and the immediate value from the instruction.

The result of the memory operation is stored in the LMD (Load Memory Data) register if it is a load instruction.

The ALU_OUT1 register is also updated with the result of the ALU operation from the previous stage (ALU_OUT), which is used in the Write Back stage for R-type and I-type instructions.

2.5.5 Write Back (WB)

In the Write Back stage, the result of the instruction is written back to the register file.

For load instructions, the data is taken from the LMD register, while for R-type and the remaining I-type instructions, the data is taken from the ALU_OUT1 register. The destination register is specified in the instruction, and the data is written to the corresponding register in the register file.

A special case is the JAL instruction, which writes the return address to the link register (R31) instead of the destination register specified in the instruction. The return address is the value of the NPC register at the end of the Memory Access stage.

2.6 Control Signals

The DLX datapath is controlled by a set of control signals that determine the operation of the ALU, the selection of registers, and the flow of data between the components. These control signals are generated by the Control Unit based on the instruction being executed.

More information on the control signals can be found in the Control Unit chapter.

2.7 Forwarding

Forwarding is a technique used in the DLX datapath to resolve data hazards that occur when an instruction depends on the result of a previous instruction that has not yet completed. Forwarding allows the datapath to use the result of a previous instruction directly, without waiting for it to be written back to the register file. This reduces the number of stalls in the pipeline and improves performance.

In particular, in the chosen implementation, the forwarding paths are designed to handle the following scenarios:

- Forwarding from the ALU output of the EX stage (ALU_OUT) to the ALU inputs of the next instruction in the pipeline. (R-type and I-type instructions)
- Forwarding from the ALU output of the MEM stage (ALU_OUT1) to the ALU inputs of the next instruction in the pipeline. (R-type and I-type instructions)
- Forwarding from the ALU output of the WB stage (ALU_OUT2) to the ALU inputs of the next instruction in the pipeline. (R-type and I-type instructions)
- Forwarding from the Load Memory Data (LMD) of the MEM stage to the ALU inputs of the next instruction in the pipeline. (Load instructions)
- Forwarding from the Load Memory Data (LMD1) of the WB stage to the ALU inputs of the next instruction in the pipeline. (Load instructions)

Forwarding is implemented for both the ALU input operands.

2.8 Stalls

Stalls are introduced in the DLX datapath to handle situations where data hazards cannot be resolved by forwarding, specifically when a load instruction is followed by an instruction that uses the loaded data. In this case, a stall is introduced in the pipeline to allow the load instruction to complete before the next instruction is executed.

2.8.1 Example

The tables below show two instruction sequences:

- **Case A:** forwarding resolves the data hazard between two arithmetic instructions.

- **Case B:** a load–use hazard requires the insertion of one stall cycle.

Table 2.1: **Case A** – Forwarding eliminates the RAW hazard

Cycle	1	2	3	4	5	6
add r1, r2, r3	IF	ID	EX	MEM	WB	
sub r4, r1, r5		IF	ID	EX	MEM	WB

Table 2.2: **Case B** – Load–use hazard, one stall inserted

Cycle	1	2	3	4	5	6	7
lw r1, 0(r2)	IF	ID	EX	MEM	WB		
add r3, r1, r4		IF	ID	Stall	EX	MEM	WB

The signal `STALL_OPERATION` is asserted when the condition in which a stall is required is met. Specifically, when the instruction in the `IR2` register (EX stage) is a load instruction and the instruction in the `IR1` register (MEM stage) is an arithmetic instruction that uses the same source register as the load.

The signal is asserted for one cycle, allowing the pipeline to stall, by inserting a NOP instruction in the EX stage and by freezing the content of the registers in the IF and ID stages.

2.9 Control Hazards

Control hazards arise when the flow of instruction execution is altered by a branch or jump instruction. In the current implementation, the pipeline handles control hazards by assuming that branches are not taken. This means the next sequential instruction is fetched and entered into the pipeline regardless of the actual outcome of the branch.

If a branch is ultimately taken or a jump is executed, the pipeline must be corrected. This is done by flushing the incorrect instructions and fetching the correct one from the target address. The following control signals manage this behavior:

- **JUMP_EN:** Asserted when a jump instruction is executed, signaling that the Program Counter (PC) should be updated to the jump target.
- **Cond:** Asserted when a branch is taken, indicating that the PC should be updated to the branch target.
- **FLUSH_MEM:** Asserted when either `JUMP_EN` or `Cond` is high. This signal triggers a pipeline flush to discard incorrect instructions.

Flushing is performed by replacing the control signals in the EX and MEM stages with NOPs, effectively canceling any instructions in those stages. This ensures that the next instruction fetched and executed is from the correct target address.

CHAPTER 3

Control Unit

3.1 Overview

The Control Unit (CU) of the DLX processor is responsible for generating control signals that manage the operations of the Datapath. It interprets the instruction fetched from memory and produces control signals for each instruction for the Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Write Back (WB) stages of the pipeline.

The CU operates in a pipelined manner, allowing it to process multiple instructions simultaneously by generating control signals for each stage of the pipeline.

3.2 Control Unit Design

The chosen implementation of the Control Unit is the Hardwired Control Unit (CU-HW), which uses combinational logic to generate control signals based on the current instruction and the state of the pipeline. The control signals are passed through the pipeline stages, ensuring that only the signals required for subsequent stages are forwarded, while signals that are no longer needed are discarded. This approach is simple and efficient and allows for fast control signal generation, but it is less flexible than a microprogrammed control unit.

3.3 Block Diagram

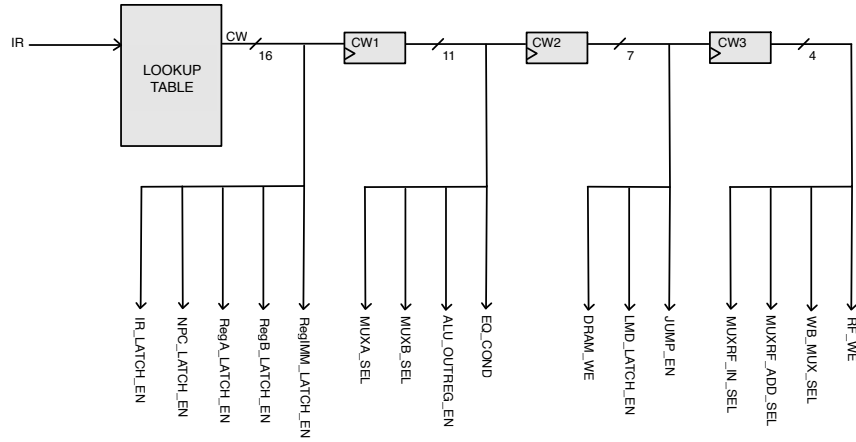


Figure 3.1: Block diagram of the DLX Control Unit.

3.4 Control Signals

The control signals are grouped by the pipeline stages they affect, and each signal is responsible for a specific operation within that stage. The control signals are defined as follows:

- **Instruction Fetch (IF) Stage:**

- IR.LATCH_EN (bit 15): Enables the latch for the Instruction Register.
- NPC.LATCH_EN (bit 14): Enables the latch for the Next Program Counter Register.

- **Instruction Decode (ID) Stage:**

- RegA.LATCH_EN (bit 13): Enables the latch for Register A.
- RegB.LATCH_EN (bit 12): Enables the latch for Register B.
- RegIMM.LATCH_EN (bit 11): Enables the latch for the Immediate Register.

- **Execution (EX) Stage:**

- MUXA_SEL (bit 10): Selects the input for MUX-A.
- MUXB_SEL (bit 9): Selects the input for MUX-B.
- ALU_OUTREG_EN (bit 8): Enables the latch for the ALU Output Register.
- EQ_COND (bit 7): Determines the branch condition based on equality.

- **Memory Access (MEM) Stage:**

- DRAM_WE (bit 6): Enables write operations to the Data RAM.
- LMD.LATCH_EN (bit 5): Enables the latch for the Load Memory Data Register.
- JUMP_EN (bit 4): Enables the jump operation for the Program Counter.

- **Write Back (WB) Stage:**

- MUXRF_IN_SEL (bit 3): Selects the input for the Register File.
- MUXRF_ADD_SEL (bit 2): Selects the address input for the Register File.
- WB_MUX_SEL (bit 1): Selects the input for the Write Back MUX.
- RF_WE (bit 0): Enables write operations to the Register File.

3.5 Lookup Table (LUT)

The CU uses a LUT to store control words for each instruction. Each control word is a bit vector that defines the control signals for a specific instruction. The control words are indexed by the opcode of the instruction, allowing the CU to quickly retrieve the appropriate control signals for each instruction. Opcodes are extracted from the instruction register (IR) and can be seen in the table 1.1.

The Lookup Table is implemented as a memory block that maps each instruction to a specific opcode. Since the DLX instruction set is fixed-length, the CU can efficiently decode the instruction and retrieve the corresponding control word from the LUT. Since the opcodes of the implemented instructions are fixed and not sequential, the LUT contains many unused entries which are initialized to NOP (no operation) control words.

3.5.1 LUT Encoding

Mnemonic	Opcode	Control Word															
		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
r-type	0x00	1	1	1	1	0	0	1	1	0	0	0	0	0	0	0	1
j	0x02	1	1	0	0	1	1	0	1	0	0	0	1	0	0	0	0
jal	0x03	1	1	0	0	1	1	0	1	0	0	0	1	1	1	0	1
beqz	0x04	1	1	1	0	1	1	0	1	1	0	0	0	0	0	0	0
bnez	0x05	1	1	1	0	1	1	0	1	1	0	0	0	0	0	0	0
addi	0x08	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
subi	0x0A	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
andi	0x0C	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
ori	0x0D	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
xori	0x0E	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
slli	0x14	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
nop	0x15	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
srli	0x16	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
snei	0x19	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
slei	0x1C	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
sgei	0x1D	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	1
lw	0x23	1	1	1	0	1	0	0	1	0	0	1	0	0	0	1	1
sw	0x2B	1	1	1	1	1	0	0	1	0	1	0	0	0	0	0	0

Table 3.1: Control Words Encoding for DLX Instructions

3.6 ALU Operation Code Generation

The CU generates the ALU operation code based on the opcode and function code of the instruction. The ALU operation code is used to control the ALU operations in the EX stage of the pipeline.

Since the Opcode is not unique for each instruction, the CU must also consider the function code for R-type instructions to determine the specific ALU operation.

3.7 Flush and Stall Operations

The CU supports flush and stall operations to handle pipeline hazards. Flush operations are used to clear the pipeline when a branch instruction is taken, ensuring that the pipeline does not execute incorrect instructions. Stall operations are used to pause the pipeline when a data hazard occurs, allowing the necessary data to be available before proceeding with the execution of the next instruction.

By operating on the control words and ALU operation codes, the CU can effectively manage the pipeline and ensure correct execution of instructions.

Care must be taken also in the Datapath counterpart, to ensure that registers have the same behavior during flush and stall operations.

3.7.1 Flush Operation

When a flush operation is triggered, the CU resets the control word at the ID and EX stages of the pipeline to zero, preventing the execution of the instruction present in that stages for the whole pipeline.

This ensures that the pipeline does not execute any instructions that may be incorrect due to a branch instruction being taken.

3.7.2 Stall Operation

When a stall operation is triggered, the CU holds the current control words and ALU operation codes present in the IF and ID stages of the pipeline, preventing the pipeline from advancing to the next instruction.

This allows the necessary data to be available before proceeding with the execution of the next instruction.

CHAPTER 4

Memories

4.1 Overview

The datapath of the DLX processor closely interacts with some memory components, each serving a specific purpose in the execution of instructions. These memories are essential for storing instructions and data and they play a crucial role in the overall functionality of the processor.

In particular, the DLX processor features the following memory components:

- **Instruction Memory (IRAM):** Stores the instructions to be executed by the processor.
- **Data Memory (DRAM):** Stores the data used by the instructions.

These memories are accessed during the instruction fetch and memory phases, allowing the processor to retrieve instructions and data as needed through specific interfaces.

These components have been described in HDL code but have not been synthesized or implemented physically.

4.2 Instruction Memory

The instruction memory (IRAM) is a crucial component of the DLX processor, responsible for storing the instructions that the processor fetches and executes. The IRAM is designed to be accessed during the instruction fetch phase, where the processor retrieves the next instruction to execute.

The IRAM is implemented as a purely combinational read-only memory (ROM) in the HDL code, which means that it is pre-loaded on reset (**RST**) with the instructions that the processor will execute. The IRAM, in this implementation, has a width of 32 bits and a depth of 48 words, which allows it to store a total of 48 instructions, each 32 bits wide.

Care must be taken to ensure that the IRAM is accessed correctly, as it is implemented as an array with an index ranging from 0 to 47. Since the PC (Program Counter) gets incremented by 4 after each instruction fetch, the IRAM must be accessed using the PC value divided by 4, through a shift operation, in order to obtain the correct address for the instruction to be fetched.

4.2.1 Block Diagram

The block diagram of the instruction memory (IRAM) is shown in Figure 4.1. The IRAM is accessed through a specific interface that allows the processor to read instructions from it.

The interface includes an address input (**ADDR**), which specifies the location of the instruction to be fetched, and a data output (**DOUT**), which provides the instruction data to the processor. There is also a reset input (**RST**) that initializes the IRAM when the processor is reset.

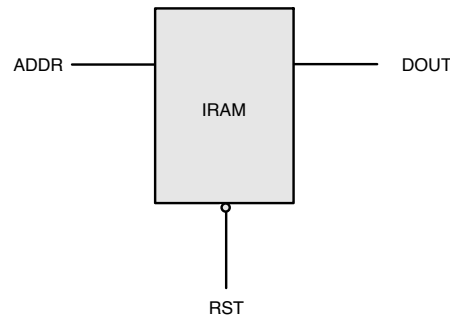


Figure 4.1: Block diagram of the DLX Instruction Memory (IRAM).

4.3 Data Memory

The data memory (DRAM) is another essential component of the DLX processor, responsible for storing the data used by the instructions. The DRAM is accessed during the memory phase, where the processor reads or writes data to it through load and store instructions.

The DRAM is designed to support sequential write operations during the memory phase, ensuring data integrity and proper synchronization with the processor's clock. For read operations, the DRAM is accessed in a purely combinational manner, allowing immediate retrieval of data without waiting for a specific clock cycle.

The DRAM has a width of 32 bits and a depth of 32 words, allowing it to store a total of 32 data words, each 32 bits wide.

4.3.1 Block Diagram

The block diagram of the data memory (DRAM) is shown in Figure 4.2.

The DRAM is accessed through a specific interface that allows the processor to read and write data to it. The interface includes an address input (**ADDR**), which specifies the location of the data to be read or written, a data input (**DATA_IN**), which provides the data to be written to the DRAM, and a data output (**DATA_OUT**), which provides the data read from the DRAM. There is also a write enable input (**WR_EN**), which allows the processor to perform write operations to the DRAM.

A clock input (**CLK**) is used to synchronize the write operations, ensuring that data is written to the DRAM at the rising edge of the clock signal.

A reset input (**RST**) is also included to initialize the DRAM when the processor is reset.

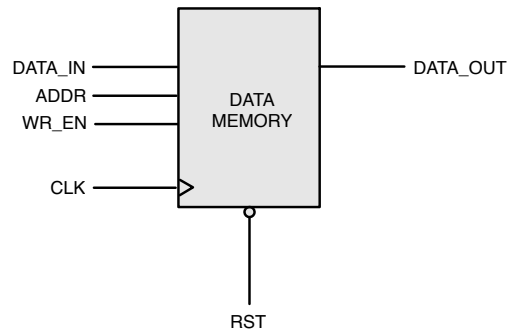


Figure 4.2: Block diagram of the DLX Data Memory (DRAM).

4.4 Memory Initialization

The instruction memory (IRAM) and data memory (DRAM) are initialized with specific values at the start of the simulation. The IRAM is pre-loaded with the instructions that the processor will execute, while the DRAM is initialized with specific data values that the instructions will operate on.

This initialization is done in the HDL code, through a process that runs at the beginning of the simulation, where a file (`test.asm.mem` for IRAM and `memContent.mem` for DRAM) containing the initial values for the IRAM and DRAM is read, and the values are loaded into the respective memories.

CHAPTER 5

Simulation

5.1 Overview

The simulation of the DLX processor, conducted using QuestaSim, is a critical step in validating the design and ensuring its functionality. It involves executing a series of predefined test programs using a specific testbench.

The testbench is a closed word component, that is, it does not interact with the outside world. It is responsible for instantiating the processor and a clock and reset signals, that are used as stimuli for the simulation.

Key components of the simulation process include the compilation of VHDL files, the assembly of test programs into machine code, and the initialization of the processor's memory. By running different test programs, the simulation evaluates the processor's handling of loops, complex scenarios, and pipeline hazards, ensuring its correctness and reliability.

5.2 Testbench Design

The testbench is designed to provide the necessary stimuli to the processor and to monitor its outputs. It includes the following components:

- **Clock Generator:** Generates a clock signal that drives the processor
- **Reset Signal:** Provides a reset signal to initialize the processor
- **DLX Processor:** Instantiates the DLX processor and connects it to the clock and reset signals

5.2.1 Block Diagram

The block diagram of the testbench is shown in Figure 5.1. It illustrates the connections between the clock generator, reset signal, and the DLX processor. The clock generator produces a clock signal that is fed into the processor, while the reset signal is used to initialize the processor at the start of the simulation.

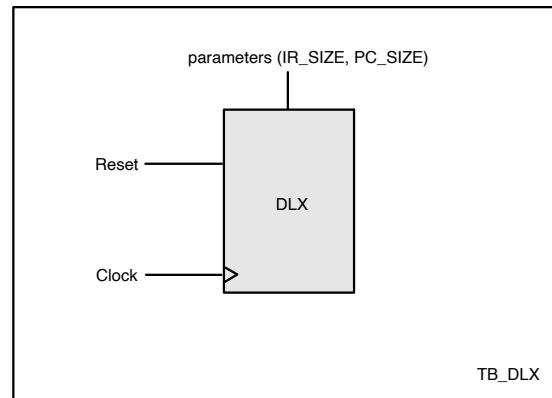


Figure 5.1: Block diagram of the DLX testbench.

5.2.2 ALU's Testbench

Due to its complexity, the ALU has its own dedicated testbench (**TB_ALU**) in order to check its proper functionality: all the possible operations are performed, each of which is tested with multiple combinations of input values.

5.3 Simulation Tool

The simulation is performed using a simulation tool that is called QuestaSim.

QuestaSim is a powerful simulation environment that supports VHDL and Verilog designs. It provides a comprehensive set of features for simulating digital designs, including waveform viewing, debugging, and performance analysis.

5.3.1 Compilation and Simulation Scripts

The simulation is automated using two scripts:

- **compile.bash**: This script compiles the VHDL files and prepares the simulation environment.
- **sim.do**: This script runs the simulation and sets up the waveform viewing.

Compilation Script

The **compile.bash** script (reported in Appendix A.1) is responsible for compiling the VHDL files and preparing the simulation environment. It includes the following steps:

- Setting up the simulation environment by sourcing the necessary scripts and creating the work library.
- Compiling the VHDL files in the correct order, ensuring that dependencies are resolved.
- Running the assembler to convert the assembly code into machine code and generate the memory file that will be used in the simulation.
- Running the simulation script to start the simulation process.

Simulation Script

The `sim.do` script (reported in Appendix A.2) is used to run the simulation and set up the waveform viewing. It includes the following steps:

- Setting the simulation time resolution and optimization flags.
- Adding waveforms to the simulation for monitoring the processor's internal signals and outputs.
- Running the simulation for a specified duration to observe the processor's behavior and outputs.

5.4 Assembler

The test programs are written in assembly language and are assembled into machine code that the DLX processor can execute.

This is done using an assembler (`dlxasm.pl`) that converts the assembly code into machine code instructions that the processor can understand. The assembler also handles the conversion of labels and symbols into addresses and offsets, ensuring that the machine code is correctly formatted for execution on the DLX processor.

5.4.1 Assembler Script

The script `assembler.bash` (reported in Appendix A.3) is used to automate the assembly process. It is called from `compile.bash` and takes an assembly file as input called `test.asm` located in the `asm_example` directory. It runs the assembler and generates a binary file named `test.asm.exe`, which is then converted into a memory file `test.asm.mem` using the `conv2memory` tool. This memory file is used in the simulation to initialize the processor's memory with the machine code instructions.

5.5 Test Programs

The processor is tested using a set of predefined test programs that cover a wide range of instructions and scenarios. These programs are designed to cover various aspects of the processor's functionality, including arithmetic operations, memory access, and control flow.

In particular, three test programs (reported in Appendix A.4) are used:

- `test1.asm`: This program tests the loop handling capabilities of the processor.
- `test2.asm`: This program tests a more complex scenario with multiple instructions, including loads and stores, and control flow.
- `test3.asm`: This program highlights stalls and forwarding in the processor's execution pipeline.

5.5.1 Test Program 1: Loop Handling

The first test program, `test1.asm`, is designed to test the loop handling capabilities of the processor. It includes a simple loop that performs arithmetic and logical operations on the registers and then jumps back to the start of the loop. The program is designed to run indefinitely, demonstrating the processor's ability to handle loops and control flow (jump instructions).

5.5.2 Test Program 2: Complex Scenario

The second test program, `test2.asm`, is more complex and includes a series of instructions that test the processor's ability to handle multiple operations, including loads and stores, a wider range of arithmetic and logical operations, and control flow (including also branches, not only jumps).

5.5.3 Test Program 3: Stalls and Forwarding

The third test program, `test3.asm`, is designed to highlight the processor's handling of stalls and forwarding in the execution pipeline. It includes a sequence of instructions that demonstrate how the processor manages data hazards and ensures correct execution order, even in the presence of stalls.

CHAPTER 6

Synthesis

6.1 Overview

This chapter provides an overview of the synthesis process for the DLX processor design. It includes details on the synthesis tool used, the synthesis results, and the optimization techniques applied to improve the design's performance and area.

6.2 Synthesis Tool

The synthesis of the DLX processor was performed using the Synopsys Design Compiler tool. This tool is widely used in the industry for synthesizing digital designs from RTL (Register Transfer Level) descriptions to gate-level netlists.

6.3 Scripts

The synthesis process was automated using a set of scripts. These scripts include a bash script for invoking the synthesis tool, two TCL scripts for running the synthesis and applying optimizations (one for the Datapath and the other for the Control Unit, which were synthesized separately), and an SDC (Synopsys Design Constraints) file for specifying timing constraints.

6.3.1 Synthesis Bash Script

The synthesis bash script (reported in Appendix B.1) is responsible for setting up the environment, creating the `results` directory, and running the TCL scripts for the Datapath and Control Unit synthesis.

6.3.2 Synthesis TCL Scripts

The synthesis TCL scripts (reported in Appendix B.2 only for the Control Unit, the one for the Datapath is almost identical) contain the commands for the synthesis and optimization of the design. The scripts perform the following tasks:

- Reading the RTL files for the Datapath or Control Unit (analyzes the HDL sources, elaborates the design and removes unconnected ports).
- Setting the design constraints, through the SDC file (Detailed in the next section).
- Applying optimizations to the design.
- Writing the synthesized netlist to a file.
- Reporting the synthesis results, including area, timing and power information.

6.3.3 Synthesis SDC File

The SDC file (reported in Appendix B.3) specifies the timing requirements for the design. It includes:

- Declares the main clock signal and sets its period, establishing timing constraints for all register-to-register paths.
- Input-to-output delay constraint: Sets a maximum allowable delay for purely combinational paths from input ports to output ports, ensuring they meet the clock timing requirements.

6.4 Synthesis Results

The synthesis results for the DLX processor design are summarized in the table below. The results include the area, timing, and power consumption of the synthesized design.

Table 6.1: Synthesis Results for DLX Processor - Datapath

Clock Period	Frequency	Area	Slack	Power		
				Dynamic	Leakage	Total
0.18 <i>ns</i>	5.55 <i>GHz</i>	17731.83 μm^2	-1.24 <i>ns</i> (VIOLATED)	60.95 <i>mW</i>	355.71 μW	61.31 <i>mW</i>
0.19 <i>ns</i>	5.26 <i>GHz</i>	17715.33 μm^2	-1.22 <i>ns</i> (VIOLATED)	57.70 <i>mW</i>	355.58 μW	58.05 <i>mW</i>
1.50 <i>ns</i>	666.67 <i>MHz</i>	14279.94 μm^2	-0.10 <i>ns</i> (VIOLATED)	7.47 <i>mW</i>	297.32 μW	7.77 <i>mW</i>
1.60 <i>ns</i>	625.00 <i>MHz</i>	14186.58 μm^2	-0.01 <i>ns</i> (VIOLATED)	6.99 <i>mW</i>	295.92 μW	7.29 <i>mW</i>
1.75 <i>ns</i>	571.43 <i>MHz</i>	13594.99 μm^2	0.00 <i>ns</i> (MET)	6.33 <i>mW</i>	282.19 μW	6.61 <i>mW</i>
2.00 <i>ns</i>	500.00 <i>MHz</i>	13381.13 μm^2	0.00 <i>ns</i> (MET)	5.52 <i>mW</i>	277.10 μW	5.80 <i>mW</i>
2.50 <i>ns</i>	400.00 <i>MHz</i>	13318.09 μm^2	0.00 <i>ns</i> (MET)	4.41 <i>mW</i>	276.06 μW	4.69 <i>mW</i>
3.00 <i>ns</i>	333.33 <i>MHz</i>	13251.32 μm^2	0.01 <i>ns</i> (MET)	3.67 <i>mW</i>	274.56 μW	3.94 <i>mW</i>
3.125 <i>ns</i>	320.00 <i>MHz</i>	13253.98 μm^2	0.02 <i>ns</i> (MET)	3.51 <i>mW</i>	274.49 μW	3.79 <i>mW</i>
3.20 <i>ns</i>	312.50 <i>MHz</i>	13250.52 μm^2	0.06 <i>ns</i> (MET)	3.43 <i>mW</i>	274.31 μW	3.70 <i>mW</i>
3.25 <i>ns</i>	307.69 <i>MHz</i>	13251.06 μm^2	0.11 <i>ns</i> (MET)	3.38 <i>mW</i>	274.31 μW	3.65 <i>mW</i>
3.50 <i>ns</i>	285.71 <i>MHz</i>	13244.40 μm^2	0.37 <i>ns</i> (MET)	3.13 <i>mW</i>	274.20 μW	3.41 <i>mW</i>
4.00 <i>ns</i>	250.00 <i>MHz</i>	13243.87 μm^2	0.86 <i>ns</i> (MET)	2.74 <i>mW</i>	274.15 μW	3.02 <i>mW</i>
5.00 <i>ns</i>	200.00 <i>MHz</i>	13260.90 μm^2	1.79 <i>ns</i> (MET)	2.20 <i>mW</i>	273.44 μW	2.47 <i>mW</i>

Table 6.2: Synthesis Results for DLX Processor - Control Unit

Clock Period	Frequency	Area	Slack	Power		
				Dynamic	Leakage	Total
0.18 <i>ns</i>	5.55 <i>GHz</i>	106.67 μm^2	-0.01 <i>ns</i> (VIOLATED)	344.88 μW	2.28 μW	347.15 μW
0.19 <i>ns</i>	5.26 <i>GHz</i>	106.40 μm^2	0.00 <i>ns</i> (MET)	297.09 μW	2.23 μW	299.32 μW
1.50 <i>ns</i>	666.67 <i>MHz</i>	100.01 μm^2	1.30 <i>ns</i> (MET)	37.02 μW	2.09 μW	39.10 μW
1.60 <i>ns</i>	625.00 <i>MHz</i>	100.02 μm^2	1.40 <i>ns</i> (MET)	34.70 μW	2.08 μW	36.79 μW
1.75 <i>ns</i>	571.43 <i>MHz</i>	100.02 μm^2	1.55 <i>ns</i> (MET)	31.73 μW	2.09 μW	33.81 μW
2.00 <i>ns</i>	500.00 <i>MHz</i>	100.02 μm^2	1.80 <i>ns</i> (MET)	27.76 μW	2.09 μW	29.85 μW
2.50 <i>ns</i>	400.00 <i>MHz</i>	100.02 μm^2	2.30 <i>ns</i> (MET)	22.21 μW	2.09 μW	24.30 μW
3.00 <i>ns</i>	333.33 <i>MHz</i>	100.02 μm^2	2.80 <i>ns</i> (MET)	18.51 μW	2.09 μW	20.59 μW
3.125 <i>ns</i>	320.00 <i>MHz</i>	100.02 μm^2	2.93 <i>ns</i> (MET)	17.77 μW	2.09 μW	19.85 μW
3.20 <i>ns</i>	312.50 <i>MHz</i>	100.02 μm^2	3.00 <i>ns</i> (MET)	17.35 μW	2.09 μW	19.44 μW
3.25 <i>ns</i>	307.69 <i>MHz</i>	100.02 μm^2	3.05 <i>ns</i> (MET)	17.08 μW	2.09 μW	19.17 μW
3.50 <i>ns</i>	285.71 <i>MHz</i>	100.02 μm^2	3.30 <i>ns</i> (MET)	15.86 μW	2.09 μW	17.95 μW
4.00 <i>ns</i>	250.00 <i>MHz</i>	100.02 μm^2	3.80 <i>ns</i> (MET)	13.88 μW	2.09 μW	15.97 μW
5.00 <i>ns</i>	200.00 <i>MHz</i>	100.02 μm^2	4.80 <i>ns</i> (MET)	11.11 μW	2.09 μW	13.19 μW

6.5 Analysis of Results

6.5.1 Timing Analysis

The timing analysis of the synthesized design shows that the Datapath can achieve a maximum frequency of 571.43 MHz with a clock period of 1.75 ns; the Control Unit, instead, can achieve a maximum frequency of 5.26 GHz with a clock period of 0.19 ns.

Of course, since they are two separate modules but they are connected in the final DLX processor, the maximum frequency of the whole design is limited by the slowest module, which is the Datapath.

In order to have some margin, and be safe in the physical design phase, the design was synthesized with a clock period of 3.25 ns, which corresponds to a frequency of 307.69 MHz and a slack of 0.11 ns for the Datapath and of 3.05 ns for the Control Unit.

6.5.2 Area Analysis

As can be seen from the tables 6.1 and 6.2, the area of the synthesized design is relatively small, with the Datapath occupying $13251.06 \mu m^2$ and the Control Unit occupying $100.02 \mu m^2$. The total area of the DLX processor is therefore $13351.08 \mu m^2$.

It is worth noting that the area of the Control Unit is significantly smaller than that of the Datapath, which is expected since the Control Unit is responsible for generating control signals and does not require as many resources as the Datapath, which performs the actual data processing. Also, while the area of the Control Unit is almost constant for all the clock periods, the area of the Datapath varies significantly with the clock period, as it is more sensitive to the timing constraints and the optimizations applied.

Another important aspect to point out is that the area of the synthesized design is quite small, especially considering that the DLX processor is a complete implementation of a RISC architecture. This is a result of the optimizations applied during the synthesis process, which reduced the area while maintaining the required performance.

6.5.3 Power Analysis

The power consumption of the synthesized design is also quite low, with the Datapath consuming 3.65 mW at the chosen clock period of 3.25 ns, and the Control Unit consuming $19.17 \mu W$ at the same clock period.

The total power consumption of the DLX processor is therefore 3.67 mW, mainly dominated by the Datapath.

The power consumption is also a result of the optimizations applied during the synthesis process, which reduced the dynamic power consumption while maintaining the required performance. The leakage power remains quite constant across different clock periods, especially for the Control Unit, as it is mainly determined by the technology used and the design's static characteristics.

CHAPTER 7

Physical Design

7.1 Overview

This chapter provides an overview of the physical design process for the DLX processor design. It includes details on the physical design tool used, the floorplan, placement, routing, and the final layout of the design.

7.2 Physical Design Tool

The physical design of the DLX processor was performed using the Cadence Innovus tool. This tool is widely used in the industry for physical design tasks such as floorplanning, placement, routing, and layout generation.

7.3 Physical Design Flow

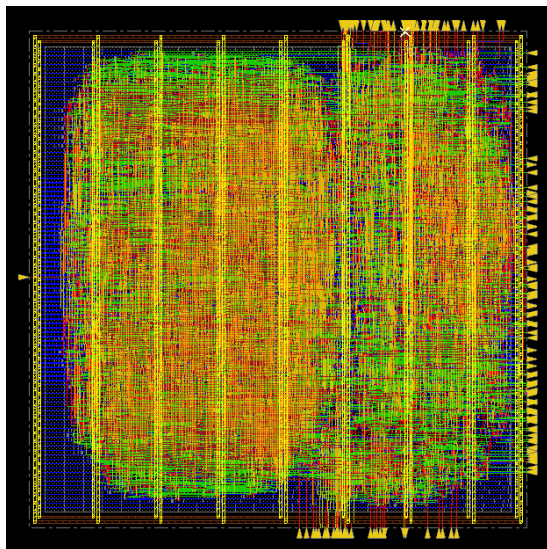
The physical design flow for the DLX processor was carried out twice: once for the Datapath and once for the Control Unit. In both cases, the steps described below were followed:

- **Configuration and Design Import:** Setting up the environment and importing the post-synthesis Verilog netlist and technology libraries.
- **Floorplanning:** Defining the floorplan for the design, including core aspect ratio, utilization, and margins.
- **Power Planning:** Creating power rings and stripes to ensure robust power delivery to the standard cells (M9, M10 layers).
- **Placement:** Placing standard cells in predefined rows, considering congestion and IO pin locations (M1-M8 layers).
- **Pre-CTS (Clock Tree Synthesis) Optimization:** Running pre-CTS optimization to perform initial timing analysis and improve timing before clock tree synthesis.
- **Clock Tree Synthesis (CTS):** Synthesizing the clock tree, inserting buffers to reduce load and delay of the clock signal, in order to meet clock transition and skew constraints, and verifying timing after CTS.
- **Post-CTS Optimization:** Further optimizing the design after CTS to address any remaining setup or hold violations and improve overall timing.
- **Filler Insertion:** Inserting filler cells to fill gaps between placed standard cells, ensuring continuity of N+ and P+ wells.

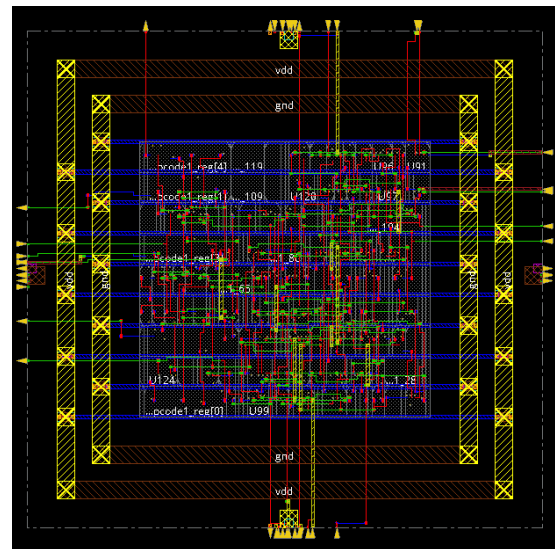
- **Routing:** Connecting all signal nets using the available metal layers, ensuring that all signal paths are routed correctly and efficiently.
- **Post-Routing Optimization:** Analyzing timing after routing and performing final optimizations to eliminate any residual violations.
- **Delay Analysis:** Involves generating reports for setup and hold violations, analyzing slack values, and debugging critical paths using Innovus functionalities.
- **Parasitic Analysis:** Extracting parasitic capacitances and resistances from the routed design to understand the impact of routing on timing and power consumption.
- **Design Analysis and Verification:** Involves checking connectivity and design rules to ensure no violations, analyzing power consumption, and saving key outputs like the post place-and-route Verilog netlist and delay annotation file (.sdf).

7.4 Schematic

The schematics of the DLX processor design after physical design are shown in Figure 7.1. The schematics include the Datapath and Control Unit, with all the standard cells placed and routed according to the physical design flow.



(a) Datapath Schematic



(b) Control Unit Schematic

Figure 7.1: Schematics of the DLX Processor Design after Physical Design

7.5 Control Unit Results

The physical design results for the Control Unit of the DLX processor are summarized below:

7.5.1 Area

The total gate area of the Control Unit is $0.7980 \mu m^2$, with the following breakdown:

- Total Gates: 136
- Total Cells: 85
- Total Area: $108.5 \mu m^2$

7.5.2 Power

The power analysis and distribution for the Control Unit were conducted pre-CTS, post-CTS, and post-route, all with their respective optimizations. The results are summarized in Table 7.1.

Table 7.1: Power Distribution for the Control Unit

Group	Power		Percentage	
	Dynamic	Leakage	Total	
Sequential	0.0090	0.0004	0.0099	36.75
Combinational	0.0080	0.0073	0.0171	63.25
Total	0.0170	0.0077	0.0270	100.00

The results indicate that the majority of the power consumption is attributed to combinational logic (63.25%), followed by sequential elements (36.75%).

The total power consumption is 0.027 mW, with the following breakdown:

- **Internal Power:** 0.017 mW (63.15%)
- **Switching Power:** 0.0077 mW (28.57%)
- **Leakage Power:** 0.0022 mW (8.28%)

Additionally, the highest average power consumption was observed in `aluOpcode1_reg[2]` at 0.0023 mW, while the highest leakage power was 0.00013 mW for the same component, which is a cell in one of the pipelined registers in charge of storing the ALU opcode.

7.5.3 Timing Analysis

The timing analysis for the Control Unit was conducted at various stages of the physical design flow, including pre-CTS, post-CTS, and post-routing, each followed by its respective optimization. The results are summarized in the tables below, which include setup and hold timing analysis for both pre-CTS and post-route optimization stages. The timing analysis was performed using the Innovus tool, which provides detailed reports on timing violations, worst negative slack (WNS), total negative slack (TNS), and the number of violating paths for different views (all, reg2reg, and default).

Table 7.2: Setup Timing – Pre-CTS

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	2.840	0.000	0	10
reg2reg	3.071	0.000	0	5
default	2.840	0.000	0	10

Table 7.3: Setup Timing – Post-CTS Opt

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	2.832	0.000	0	10
reg2reg	3.073	0.000	0	5
default	2.832	0.000	0	10

Table 7.4: Hold Timing – Pre-CTS

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	-0.184	-0.805	5	10
reg2reg	0.077	0.000	0	5
default	-0.184	-0.805	5	10

Table 7.5: Hold Timing – Post-CTS Opt

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	0.006	0.000	0	10
reg2reg	0.077	0.000	0	5
default	0.006	0.000	0	10

The timing analysis shows that the Control Unit meets the setup and hold requirements after post-routing optimization, with no violating paths. The setup WNS (Worst Negative Slack) is 2.832 ns, indicating that the design has sufficient timing margin. The hold WNS is 0.006 ns, which is also satisfactory, as it indicates that the design meets hold requirements.

7.6 Datapath Results

The physical design results for the Datapath of the DLX processor are summarized below:

7.6.1 Area

The total gate area of the Datapath is $0.7980 \mu m^2$, with the following breakdown:

- Total Gates: 16828
- Total Cells: 6598
- Total Area: $13429.0 \mu m^2$

7.6.2 Power

The power analysis and distribution for the Datapath were conducted pre-CTS, post-CTS, and post-route, all with their respective optimizations. The results are summarized in Table 7.6.

Table 7.6: Power Distribution for the Datapath

Group	Power (mW)			Percentage
	Dynamic	Leakage	Total	
Sequential	3.0606	0.1222	3.183	53.51
Combinational	1.7894	0.1432	1.933	32.50
Clock (Combinational)	0.8308	0.00107	0.8319	13.99
Total	5.6808	0.2665	5.9475	100.00

The results indicate that the majority of the power consumption is attributed to sequential elements (53.51%), followed by combinational logic (32.50%) and clock combinational logic (13.99%).

The total power consumption is 5.9475 mW, with the following breakdown:

- **Internal Power:** 3.5498 mW (59.69%)

- **Switching Power:** 2.1312 mW (35.83%)
- **Leakage Power:** 0.2665 mW (4.48%)

Additionally, the highest average power consumption was observed in `CTS_cc1_a_buf_00117` at 0.02482 mW, while the highest leakage power was 8.656e-05 mW for `NPC_reg[4]`.

7.6.3 Timing Analysis

The timing analysis for the Datapath was conducted at various stages of the physical design flow, including pre-CTS, post-CTS, and post-routing, each followed by its respective optimization. The results are summarized in the tables below, which include setup and hold timing analysis for both pre-CTS and post-route optimization stages.

Table 7.7: Setup Timing – Pre-CTS

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	0.458	0.000	0	1975
reg2reg	0.458	0.000	0	1520
default	1.455	0.000	0	1703

Table 7.8: Setup Timing – Post-Route Opt

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	0.917	0.000	0	1975
reg2reg	0.917	0.000	0	1520
default	1.632	0.000	0	1703

Table 7.9: Hold Timing – Pre-CTS

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	-0.530	-144.314	422	1942
reg2reg	0.075	0.000	0	1520
default	-0.530	-144.314	422	1670

Table 7.10: Hold Timing – Post-Route Opt

View	WNS (ns)	TNS (ns)	Viol. Paths	All Paths
all	0.007	0.000	0	1942
reg2reg	0.079	0.000	0	1520
default	0.007	0.000	0	1670

The timing analysis shows that the Datapath meets the setup and hold requirements after post-routing optimization, with no violating paths. The setup WNS (Worst Negative Slack) is 0.917 ns, indicating that the design has sufficient timing margin. The hold WNS is 0.007 ns, which is also satisfactory, as it indicates that the design meets hold requirements.

7.7 Conclusion

In conclusion, the physical design of the DLX processor, including both the Control Unit and Datapath, has been successfully completed with satisfactory results in terms of area, power consumption, and timing. The design meets the required specifications and is ready for further verification and testing.

7.8 Comparison of Synthesis vs Physical Design Results

This section compares the power, timing, and area results obtained during the synthesis stage with those achieved after physical design for both the Control Unit and Datapath of the DLX processor.

7.8.1 Control Unit Comparison

- **Area:** The synthesized area of the Control Unit was smaller compared to the post-physical design area due to the addition of filler cells, power routing structures, and clock buffers during physical design. The synthesized area was approximately $100 \mu m^2$, whereas the physical design area increased to $108.5 \mu m^2$.

- **Power:** The power consumption increased slightly after physical design due to the inclusion of parasitic effects and additional routing capacitances. The synthesized power was 0.019 mW, while the physical design power was 0.027 mW.
- **Timing:** The Control Unit exhibited no timing violations during synthesis, with a slack of 3.05 ns. After physical design optimizations, the timing margin slightly decreased, showing a setup WNS of 2.832 ns and a hold WNS of 0.006 ns, ensuring compliance with all timing constraints.

7.8.2 Datapath Comparison

- **Area:** Similar to the Control Unit, the Datapath area increased after physical design due to the addition of filler cells and routing structures. The synthesized area was approximately $13251 \mu m^2$, whereas the physical design area increased to $13429 \mu m^2$.
- **Power:** The power consumption increased after physical design due to parasitic effects and routing capacitances. The synthesized power was 3.65 mW, while the physical design power was 5.9475 mW.
- **Timing:** The Control Unit exhibited no timing violations during synthesis, with a slack of 0.11 ns. After physical design optimizations, the timing margin improved, showing a setup WNS of 0.917 ns and a hold WNS of 0.007 ns, ensuring compliance with all timing constraints.

7.8.3 Summary

The comparison highlights the impact of physical design on the final results:

- Physical design increases area and power due to the inclusion of routing structures, parasitic effects, and filler cells.
- Design optimizations during physical design ensure that timing constraints are met, even if the timing margins are in some cases slightly reduced compared to synthesis.

Overall, physical design ensures the synthesized design is transformed into a manufacturable layout while meeting all design constraints.

CHAPTER 8

Conclusions and Further Implementations

8.1 Overview

This chapter summarizes the key findings of the DLX processor design project. It also discusses potential future work and enhancements that could be made to the design.

8.2 Key Findings

The DLX processor design project has successfully demonstrated the capabilities of the DLX architecture in implementing a simple yet effective RISC processor. The design has been synthesized and physically implemented, showing that it meets the specified timing constraints and operates correctly for the intended applications. The project has highlighted the importance of using industry-standard tools and methodologies for digital design, including synthesis, physical design, and verification. The use of the DLX architecture has provided a solid foundation for understanding the principles of RISC design and the trade-offs involved in processor design.

8.3 Further Implementations

While the current design meets the basic requirements of a DLX processor, there are several areas for potential enhancement and further implementation:

- **Extended Instruction Set:** The current design implements a basic instruction set. Future work could involve extending the instruction set to include additional instructions, such as floating-point operations, subroutines, and more complex arithmetic operations like multiplications and divisions. This would require modifications to the control unit and datapath to support the new instructions.
- **Windowed Register File:** Along with the implementation of subroutines, a windowed register file could improve the performance of the processor by allowing for more efficient handling of function calls and returns. This would involve designing a register file that can dynamically allocate registers for different function calls, reducing the need for frequent saving and restoring of registers.
- **Branch Prediction:** Implementing branch prediction techniques could significantly improve the performance of the processor by reducing the number of pipeline stalls caused by branch instructions. This could involve incorporating advanced branch prediction mechanisms, such as a Branch History Table (BHT) to track the outcomes of previous branches and improve prediction accuracy, or a Branch Target Buffer (BTB) to store and quickly retrieve the target addresses of branch instructions, enabling speculative execution along the predicted path.
- **Performance Optimization:** Further optimizations could be made to improve the performance of the processor, such as superscalar execution, or out-of-order execution. These techniques could

significantly increase the throughput of the processor and reduce the execution time of programs, at the cost of increased complexity in the control unit and datapath.

- **Testbench Enhancements:** The current testbench could be extended to perform assertions and checks to verify the correctness of the design more thoroughly. This could include checking for specific conditions, such as ensuring that the output of certain instructions matches expected values, or verifying that the processor behaves correctly under various conditions.
- **Synthesis optimization improvements:** The synthesis scripts could be enhanced to include more advanced optimization techniques to reduce area, improve timing, or reduce power consumption. This could involve using more aggressive optimization settings, applying specific optimization techniques like LVT (Low Voltage Threshold) cells for critical paths and HVT (High Voltage Threshold) cells for non-critical paths, or using additional synthesis tools to achieve better results.
- **Physical Design Automation:** The physical design process could be automated using scripts that handle the floorplanning, placement, routing, and optimization steps, reducing or removing the need for manual intervention.
- **Testing and Verification:** Further testing and verification of the design could be performed to ensure its correctness and reliability. This could include extensive simulation or formal verification techniques to prove the correctness of the design against its specifications.

APPENDIX A

Simulation

A.1 Subappendix 1: Compilation Script

Listing A.1: Compilation Bash Script

```
source /eda/scripts/init_questa_core_prime
vlib work

vcom 000-globals.vhd
vcom 02-constants.vhd

vcom a.a-CUHW.vhd

vcom a.b.a-alu/a.b.a.a.a.a-g_block.vhd
vcom a.b.a-alu/a.b.a.a.a.b-pg_block.vhd
vcom a.b.a-alu/a.b.a.a.a-carry_generator_generic.vhd
vcom a.b.a-alu/a.b.a.a.b.a.a.a-fa.vhd
vcom a.b.a-alu/a.b.a.a.b.a.a-rca.vhd
vcom a.b.a-alu/a.b.a.a.b.a.b-mux21_generic.vhd
vcom a.b.a-alu/a.b.a.a.b.a-carry_select_block_generic.vhd
vcom a.b.a-alu/a.b.a.a.b-sum_generator_generic.vhd
vcom a.b.a-alu/a.b.a.a-p4_adder.vhd
vcom a.b.a-alu/a.b.a.b-comparator.vhd
vcom a.b.a-alu/a.b.a.c.a-logicals_bit.vhd
vcom a.b.a-alu/a.b.a.c-logicals.vhd
vcom a.b.a-alu/a.b.a.d-shifter.vhd

vcom a.b.a-alu.vhd

# vcom test_bench/TB_ALU.vhd
# vsim -do sim_ALU.do

vcom a.b.b-registerfile.vhd

vcom a.b-DataPath.vhd
vcom a.c-IRAM.vhd
vcom a.d-data_memory.vhd

vcom a-DLX.vhd
```

```
vcom test_bench/TB_DLX.vhd
```

```
bash ../assembler.bash
```

```
vsim -do sim.do
```

A.2 Subappendix 2: Simulation Script

```
vsim -t 10ps work.CFG_TB -voptargs=+acc
add wave *
add wave -position insertpoint sim:/tb_dlx/U1/CU_I/cw
add wave -position insertpoint sim:/tb_dlx/U1/CU_I/cw1
add wave -position insertpoint sim:/tb_dlx/U1/CU_I/cw2
add wave -position insertpoint sim:/tb_dlx/U1/CU_I/cw3
add wave -position insertpoint sim:/tb_dlx/U1/CU_I/aluOpcode_i

add wave -position insertpoint sim:/tb_dlx/U1/DATAMEM/DMEM
add wave -position insertpoint sim:/TB_DLX/U1/IRAM_I/IRAM_mem
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/REG_FILE/REGISTERS

add wave -position insertpoint sim:/tb_dlx/U1/DP_I/STALL_OPERATION
add wave -position insertpoint sim:/tb_dlx/U1/FLUSH_MEM_i

add wave -position insertpoint sim:/tb_dlx/U1/DP_I/PC
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/NPC
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/IR_IN
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/RF_A_OUT
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/RF_B_OUT
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/RegIMM
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/ALU_OUT
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/ALU_OUT1
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/ALU_OUT2
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/Cond
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/B_OUT1
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/LMD
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/LMD1

add wave -position insertpoint sim:/tb_dlx/U1/DP_I/A_OUT
add wave -position insertpoint sim:/tb_dlx/U1/DP_I/B_OUT

run 100 ns
```

A.3 Subappendix 3: Assembler Script

```
#!/usr/bin/env bash
```

```
# name of the assembly file
```

```
asm=test.asm
```

```
# get the directory of this script
```

```
d=$(cd -- "$(dirname -- "$0")" && pwd)
```

```
# run the perl scripts to assemble the DLX assembly code
perl "$d/assembler.bin/dlxasm.pl" "$d/asm_example/$asm"

# convert the binary file to a memory file
"$d/assembler.bin/conv2memory" "$d/asm_example/$asm.exe" > "$d/DLX_vhd/$asm
.mem"
```

A.4 Subappendix 4: Test Programs

A.4.1 Test Program 1

Listing A.2: Test program 1

```
; TEST-1: LOOP HANDLING

lw r25, 1(r0)      ; Load word: r25 <- memory[r0 + 1]
sne r20, r25, r20  ; Set r20 = (r25 != r20) ? 1 : 0 - set r20 based on a
                    ; comparison

myloop:
add r1, r1, r25    ; r1 <- r1 + r25
add r1, r25, r1    ; r1 <- r25 + r1
add r1, r1, r1     ; r1 <- r1 + r1
xor r20, r20, r25  ; r20 <- r20 XOR r25 - toggle LSB of r20 if r25=1
j myloop          ; Jump back to 'myloop' label (infinite loop)
```

A.4.2 Test Program 2

Listing A.3: Test program 2

```
; TEST-2: COMPLEX SCENARIO

lw r1, 4(r0)      ; Load word: r1 <- memory[r0 + 4]
lw r2, 1(r0)      ; Load word: r2 <- memory[r0 + 1]
lw r3, 22(r0)     ; Load word: r3 <- memory[r0 + 22]
lw r4, 4(r1)      ; Load word: r4 <- memory[r1 + 4]
lw r5, 3(r2)      ; Load word: r5 <- memory[r2 + 3]

loop:
add r6, r1, r1    ; r6 <- r1 + r1
addi r7, r6, #4   ; r7 <- r6 + 4
xori r8, r6, #1   ; r8 <- r6 xor 1
subi r1, r1, #1   ; r1 <- r1 - 1 (decrement loop counter)
bnez r1, loop     ; Branch to loop if r1 != 0

addi r8, r6, #2   ; r8 <- r6 + 2
addi r9, r6, #3   ; r9 <- r6 + 3
beqz r2, #0       ; Branch to 0 if r2 == 0 (not happening)

sw 0(r7), r9      ; Store word: memory[r7 + 0] <- r9
sw 0(r8), r9      ; Store word: memory[r8 + 0] <- r9
sw 0(r9), r2      ; Store word: memory[r9 + 0] <- r2
```

A.4.3 Test Program 3

Listing A.4: Test program 3

; TEST-3: HIGHLIGHT ON STALLS AND FORWARDING

```
lw r1, 4(r0)      ; Load word: r1 <- memory[r0 + 4]
lw r2, 1(r0)      ; Load word: r2 <- memory[r0 + 1]
lw r3, 22(r0)     ; Load word: r3 <- memory[r0 + 22]
lw r4, 4(r1)      ; Load word: r4 <- memory[r1 + 4]
lw r5, 3(r2)      ; Load word: r5 <- memory[r2 + 3]

addi r6, r5, #9    ; r6 = r5 + #9 (RAW on r5, STALL)
and r7, r5, r6     ; r7 = r5 and r6 (RAW on r5 and r6)
subi r8, r5, #2    ; r8 = r5 - #2 (RAW on r5)
or r9, r6, r5      ; r9 = r5 or r6 (RAW on r5 and r6)

sw 1(r7), r7       ; Store word: memory[r7 + 1] <- r7 (RAW on r7)
sw 1(r8), r8       ; Store word: memory[r8 + 1] <- r8 (RAW on r8)
sw 1(r9), r9       ; Store word: memory[r9 + 1] <- r9 (RAW on r9)
```

APPENDIX B

Synthesis Scripts

B.1 Subappendix 1: Synthesis Script

Listing B.1: Synthesis Bash Script

```
source /eda/scripts/init_design_vision

mkdir ./results

# design_vision -f syn-CU.tcl
dc_shell-xg-t -f syn-CU.tcl

# design_vision -f syn-DP.tcl
dc_shell-xg-t -f syn-DP.tcl
```

B.2 Subappendix 2: Synthesis TCL Script

Listing B.2: Synthesis TCL Script

```
#####
##
## READ DESIGN
##
#####

# ANALYZE HDL SOURCES
analyze -library WORK -format vhd1 {000-globals.vhd}
analyze -library WORK -format vhd1 {02-constants.vhd}
analyze -library WORK -format vhd1 {a.a-CU_HW.vhd}

# ELABORATE DESIGN
elaborate DLX_CU -architecture dlx_cu_hw -library WORK

# REMOVE UNCONNECTED PORTS
remove_unconnected_ports [get_cells -hierarchical *]

#####
##
## SET DESIGN CONSTRAINTS
```

```

##
#####
source ". /syn.sdc"

#####
##
## OPTIMIZE DESIGN
##
#####

# Flatten hierarchy to improve optimization freedom
ungroup -all -flatten

compile_ultra

#####
##
## SAVE DESIGN
##
#####

write -hier -format vhdl -output ". /results /DLX-CU_post_syn.vhd"
write -hier -format verilog -output ". /results /DLX-CU_post_syn.v"
write_sdc -version 1.3 ". /results /DLX-CU_post_syn.sdc"

#####
##
## WRITE REPORTS
##
#####

report_area > ". /results /area_DLX-CU.txt"
report_timing > ". /results /timing_DLX-CU.txt"
report_power > ". /results /power_DLX-CU.txt"

#####
##
## EXIT
##
#####

exit

```

B.3 Subappendix 3: Synthesis SDC Script

Listing B.3: Synthesis SDC Script

```

# Define the clock name and period
set clockName "Clk"
set clockPeriod 3.25

```

```
# Set wire load model to calculate wire capacitance for delay estimations  
set_wire_load_model -name 5K_hvratio_1_4
```

```
# Create a clock constraint with the specified period and name  
create_clock -period $clockPeriod $clockName
```

```
# Set the maximum delay for paths from inputs to outputs, ensuring that the  
  design meets timing requirements  
set_max_delay $clockPeriod -from [all_inputs] -to [all_outputs]
```